

Routes and Controllers

HTTP method + path → controller method → response.

Learning Goals

- Enable API routes and generate a controller with Artisan.
- Connect an HTTP method and path to a controller method in `routes/api.php`.
- Verify routes with `route:list` and test GET and POST requests with `curl`.

API Routes

Before this exercise, enable API routes.

```
php artisan install:api
```

This command will make `routes/api.php` automatically.

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');
```

It also modifies the `bootstrap/app.php` to add the route.

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        api: __DIR__.'/../routes/api.php', // Added
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
```

This prompt appears because API setup adds Sanctum's token-table migration.

One new database migration has been published. Would you like to run all pending database migrations? (yes/no)
[yes]:

- Choose `yes` to apply it now, or `no` and run `php artisan migrate` later.

If you don't want the interaction, use this command instead:

```
php artisan install:api --no-interaction --without-migration-prompt
```

We can use `artisan route:list` command to check the available APIs.

```
php artisan route:list --path=api
```

- We just created one API (api GET method) using `artisan install` command.

```
GET|HEAD      api/user ..... routes/api.php:6  
Showing [1] routes
```

This module uses `routes/api.php` : API middleware and the `/api` prefix, for example `/api/student` .

- `routes/api.php` is not generated with `run1-2.sh` script, we should use `php artisan` to make it.
- Laravel also has `routes/web.php` for HTML pages. We do not use it until the optional Lesson 6.

Create a Controller

The next step is to generate the controller:

```
php artisan make:controller Api/StudentController
```

Code skeleton is created automatically.

```
Controller [app/Http/Controllers/Api/StudentController.php] created successfully
```

Edit `app/Http/Controllers/Api/StudentController.php`.

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class StudentController extends Controller
{
    //
}
```

- For this lesson, use fixed data so you can focus on routing.

Return One Student

```
namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;

class StudentController extends Controller
{
    public function show()
    {
        return response()->json(['name' => 'John Doe']);
    }
}
```

In this example, we create the `show` method that can be used in the routes.

Connect the Route

In `routes/api.php` :

```
use Illuminate\Support\Facades\Route;  
use App\Http\Controllers\Api\StudentController;  
  
Route::get('/student', [StudentController::class, 'show']);
```

- We can process `GET /api/student` API .
- In this example, `GET /api/student` API calls the controller function `StudentController::show()` through the `routes/api.php` .

Process POST student API

We processed GET student API with the StudentController and routes/api.

- Likewise, we can process any APIs in a similar way.
- In this example, we process POST student API.

1. Step1: Add this method in the StudentController:

```
public function store(Request $request)
{
    $data = $request->validate(['name' => 'required|string']);
    return response()->json(['received' => $data]);
}
```

2. Step 2: Register `Route::post('/student',`
`[StudentController::class, 'store']);` in the
`routes/api.php` .

- This exercise echoes input; it does not save a student yet.

Test the Connection

Using `php artisan route:list` command, we can check the APIs are properly registered.

```
> php artisan route:list --path=api
```

```
GET|HEAD    api/student ..... Api\StudentController@show
POST        api/student ..... Api\StudentController@store
GET|HEAD    api/user ..... routes/api.php:7
```

Run the Laravel server using `php artisan`.

```
php artisan serve --host=0.0.0.0 --port=8080 // linux
php artisan serve --port=8080 // mac
```

Run curl to access the APIs

```
curl -i -H 'Accept: application/json' \  
  http://localhost:8080/api/student  
curl -i -X POST http://localhost:8080/api/student \  
  -H 'Accept: application/json' \  
  -H 'Content-Type: application/json' -d '{"name":"Jane"}'
```

Repeat POST with `{}` and inspect the validation response.

What run1-3.sh Sets Up

1. Creates a fresh Laravel project.

```
composer create-project laravel/laravel
```

No database yet — this lesson deliberately uses fixed data in the controller so you can focus on routing before adding persistence.

2. Enables API routing (creates `routes/api.php`)

```
php artisan install:api --no-interaction --without-migration-prompt
```

3. Copy the Controller and the Routes

You generated the controller with this command:

```
php artisan make:controller Api/StudentController
```

The script copies the finished files instead:

```
mkdir -p app/Http/Controllers/Api  
cp "$SOURCE_DIR/app/Http/Controllers/Api/StudentController.php"  
  app/Http/Controllers/Api/StudentController.php  
cp "$SOURCE_DIR/routes/api.php" routes/api.php
```

- The generated controller is empty, so it would need editing anyway.
- `make:controller` will not overwrite an existing file, so a rerun could not update it.
- `cp` replaces the `routes/api.php` made by `install:api` with one that registers the routes.
- `cp` does not create folders, so `mkdir -p` comes first.

APIs

- Read `student-api1-3/app/Http/Controllers/Api/StudentController.php` to find all the available APIs for this project.
- Read `student-api1-3/routes/api.php` to find how the controller methods are connected with the routes.

Test the server

Use `run1-3.sh` with `student-api1-3/` in a separate directory.

- Trace GET and POST to their controller methods.
- Change the response and call the endpoint again.
- Explain why the data is not persistent yet.

1. Step 1: Start the server

```
php artisan serve --port=8080 // Mac
php artisan serve --host=0.0.0.0 --port=8080 // WSL
```

INFO Server running on [http://127.0.0.1:8080].

Press Ctrl+C to stop the server

Check the available APIs

```
> php artisan route:list --path=api
```

POST	api/greet	Api\StudentController@greet
GET HEAD	api/hello	Api\StudentController@hello
GET HEAD	api/student	Api\StudentController@show
POST	api/student	Api\StudentController@store
PUT	api/student	Api\StudentController@update
DELETE	api/student	Api\StudentController@destroy
GET HEAD	api/student/{id}	Api\StudentController@showById
GET HEAD	api/time	Api\StudentController@time

Showing [8] routes

2. Step 2: Access the server

```
> curl http://localhost:8080/api/hello  
{"message":"Hello World!","type":"API Response"}%
```

Check the message from Laravle

```
2026-09-29 17:42:04 /api/hello .....
```

Artisan Command Summary

Create the Laravel project, enable API routing, and generate the API controller:

```
TARGET_DIR="student-api"
```

```
composer create-project laravel/laravel "$TARGET_DIR"  
php artisan install:api --no-interaction --without-migration-prompt  
php artisan make:controller Api/StudentController
```

List the API routes, then start the development server for your environment:

```
php artisan route:list --path=api
# macOS
php artisan serve --port=8080
# Linux / WSL2
php artisan serve --host=0.0.0.0 --port=8080
```